

1. Título y Resumen

Título: Implementación de un Servo-motor de alta precisión mediante STM32 y FreeRTOS con arquitectura modular orientada a tareas.

Resumen: El proyecto consiste en el desarrollo de un **servo-motor** utilizando un motor paso a paso (simil NEMA 17), un encoder magnético AS5600 y un driver DRV8825. El sistema corre sobre una placa STM32F411 Discovery utilizando FreeRTOS. La arquitectura se basa en el desacoplamiento de funciones mediante tareas independientes: adquisición de datos por DMA (**Sensor_Task**), procesamiento de control (**Control_Task**), actuación de hardware (**Driver_Task**) y una interfaz de comunicación **USB-HID** dividida en tareas de entrada (**Input_HID_Task**) y salida (**Output_HID_Task**) para asegurar el determinismo. El sistema garantiza la recuperación automática de la posición ante perturbaciones externas, priorizando el determinismo del lazo cerrado.

2. Objetivos del Proyecto

Objetivo General

Implementar un **servo-motor de posición** basado en un motor paso a paso, controlado mediante una interfaz de consola USB-HID, que garantice precisión mecánica y estabilidad dinámica mediante el cumplimiento estricto de plazos de ejecución en un entorno de tiempo real (FreeRTOS).

Objetivos Específicos (Metas Medibles)

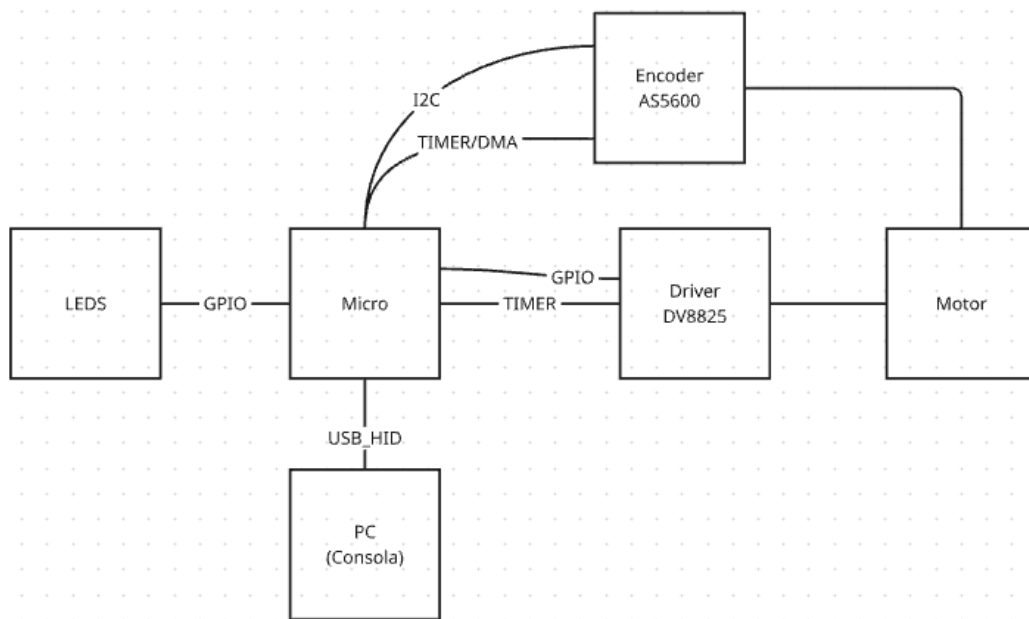
Para asegurar el funcionamiento correcto y completo, el sistema debe cumplir con los siguientes indicadores:

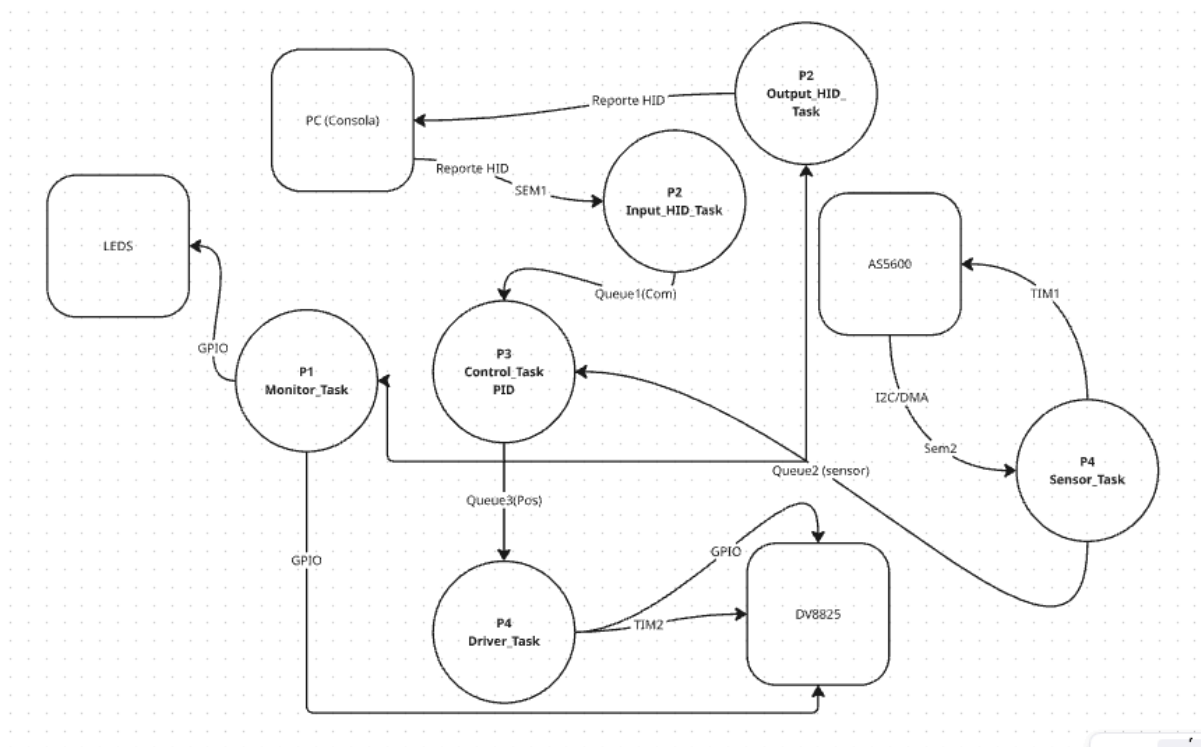
- **Determinismo en la Adquisición:** Garantizar que la lectura del sensor AS5600 se realice cada 1 ms utilizando hardware (Timer+DMA), eliminando el jitter inducido por el planificador de tareas.
- **Precisión del Control:** Lograr un error de estado estacionario menor a 1.8° en la posición absoluta, incluso ante perturbaciones manuales externas.
- **Latencia de Respuesta:** Asegurar que el tiempo transcurrido desde la llegada de un comando USB-HID hasta la actualización del movimiento del motor no supere los 5ms.
- **Eficiencia de Concurrencia:** Mantener el lazo de control PID operando a una frecuencia constante de 1kHz sin que la carga de comunicación USB o las tareas de monitoreo provoquen desbordamientos de tiempo (*deadlines* perdidos).
- **Optimización Dinámica:** Implementar una lógica de microstepping que conmute automáticamente entre paso completo (para alta velocidad), 1/2 y 1/4 de paso (para alta precisión) según la magnitud del error de posición.
- **Seguridad y Supervisión:** Detectar una condición de bloqueo mecánico (*stall*) en menos de 2 segundos y proceder al apagado seguro del driver junto con la señalización visual de error.

3. Descripción Funcional del Sistema

El sistema opera como un servo-motor de lazo cerrado. La PC envía una consigna mediante USB-HID; el sistema adquiere la posición real vía sensor magnético, procesa el error mediante un algoritmo PID y actúa sobre el driver mediante señales de pulso y configuración de pasos.

- **Entradas:** Sensor AS5600 (I2C/DMA), Consigna de usuario (USB-HID Input).
- **Salidas:** Driver DRV8825 (STEP vía Timer, DIR y M0-M2 vía GPIO), Telemetría (USB-HID Output), LEDs de estado(vía GPIO).





4. Requisitos y Restricciones de Tiempo Real

- **Hard Real-Time:** El lazo conformado por las tareas **Sensor**, **Control** y **Driver** debe completar su ciclo en un tiempo razonable para mantener la estabilidad mecánica.
- **Determinismo de Comunicación:** Se separa la tarea de recepción de comandos de la tarea de envío de telemetría. Esto asegura que un procesamiento pesado en los reportes entrantes no bloquee el flujo de datos hacia la PC.
- **Aislamiento de Periféricos:** El uso de DMA para el sensor permite que la CPU solo intervenga cuando el dato ya está disponible en RAM, minimizando el tiempo de bloqueo.

5. Arquitectura de Software y Diseño en FreeRTOS

Se implementa una estructura de 6 tareas principales interconectadas por Colas (Queues) y Semáforos:

1. **Sensor_Task:** Espera el **Sem2** (DMA Transfer Complete). Al activarse, toma el dato del AS5600 y lo envía por la **Queue2 (sensor)**.
2. **Input_HID_Task:** Se activa mediante el **Sem1** cuando llega un reporte desde la PC. Procesa la consigna y la envía por la **Queue1 (Com)**.
3. **Control_Task (PID):** El núcleo del sistema. Recibe datos de las colas de sensor y comunicación. Calcula la corrección y envía la orden a la **Queue3 (Pos)**.
4. **Driver_Task:** Recibe la orden de la **Queue3** y actualiza físicamente el **TIM2** (pasos) y los **GPIO** (dirección y microstepping).

5. **Output_HID_Task:** Recibe la posición actual desde la tarea de sensor/control y genera el reporte HID de salida hacia la PC.
6. **Monitor_Task:** Tarea de baja prioridad encargada de gestionar los LEDs y diagnósticos secundarios.

6. Arquitectura de Hardware

Microcontrolador: STM32F411 Discovery (Cortex-M4).

Periféricos y Configuración:

- **I2C1 + DMA:** Configurado para la adquisición automática de los registros de posición absoluta del sensor magnético AS5600.
- **TIM1 (Sampling):** Timer de hardware configurado para generar un evento de disparo cada 1ms, iniciando la transferencia I2C por DMA de forma determinística.
- **TIM2 (Step Generation):** Generador de pulsos de frecuencia variable, encargado de comandar el movimiento del driver DV8825.
- **USB OTG FS (HID Class):** Interfaz de comunicación configurada con reportes de interrupción de 64 bytes para asegurar una latencia fija en el intercambio de datos con la PC.
- **GPIOs de Control del Driver:**
 - **DIR:** Salida digital para definir el sentido de giro.
 - **ENABLE:** Pin crítico gestionado por la **Monitor_Task** para desenergizar el motor en caso de falla o atasco.
 - **M0, M1, M2:** Salidas digitales para el control dinámico de la resolución de micropasos (Microstepping).
- **GPIOs de Interfaz:**
 - **Red LED:** Salida configurada para señalización de errores (parpadeo en caso de *Stall* detectado por el monitor).
 - **Green/Orange LEDs:** Indicadores de estado del sistema y actividad del enlace USB.

7. Metodología de Validación y Pruebas

1. **Validación de Microstepping:** Verificar mediante osciloscopio que el cambio de resolución en los pines M0-M2 se realiza correctamente antes de iniciar una ráfaga de pasos de alta precisión.
2. **Prueba de Seguridad (Stall Detection):** Se bloqueará mecánicamente el eje del motor para simular un atasco. Se verificará que la **Monitor_Task** detecte la discrepancia entre el sensor y la consigna, desactive el driver mediante el pin de ENABLE luego de 2 segundos bloqueado y active la señalización de alerta en los LEDs.
3. **Prueba de Recuperación:** Forzar un movimiento manual y observar cómo el sistema ajusta la resolución y velocidad para retornar al punto de consigna.